Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Student Name	Ajay	Reg. No.:
Source Code	https://github.com/tonypanda7/DeepLearning_Lab	

1. Objective

The objective of this experiment is to build an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modelling on MNIST. Four models are implemented and compared under one controlled protocol: a fully connected autoencoder (FC-AE), a convolutional autoencoder (CAE), a denoising convolutional autoencoder (DCAE) and a variational autoencoder (VAE) with a two-dimensional latent space. Reconstruction is evaluated with MSE, MAE and SSIM, and the latent spaces are studied through visualisation, random sampling and interpolation.

2. Source Code

The complete implementation for this experiment (FC-AE, CAE, DCAE, VAE, all metrics and plots) is available at:

https://github.com/tonypanda7/DeepLearning_Lab

The experiment was run as a single Kaggle notebook with GPU acceleration (Tesla T4), using TensorFlow/Keras, NumPy, pandas, Matplotlib and scikit-image.

3. Dataset and Experimental Setup

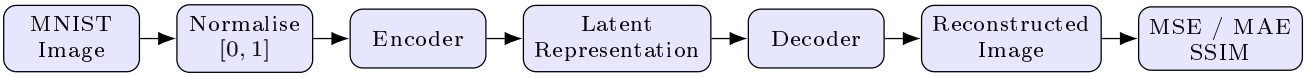
Dataset: MNIST handwritten digits (grayscale, $28 \times 28 \times 1$), loaded through `keras.datasets.mnist`. Pixel values were scaled from $[0, 255]$ to $[0, 1]$. Following the recommended laboratory subset, the first 10,000 training images and the first 2,000 test images were used. Ten percent of the training subset (1,000 images) was held out as a validation set, so the 2,000 test images are never seen during training or model selection. Labels are used only for visualisation (latent-space colouring and high-error captions), never for training. The FC-AE receives flattened vectors ($x \in \mathbb{R}^{784}$); all other models receive $x \in \mathbb{R}^{28\times28\times1}$.

Setting	Value
Training / validation / test images	9,000 / 1,000 / 2,000
Optimizer, learning rate	Adam, 10^{-3}
Batch size	128
Epochs	20 (FC-AE, CAE, DCAE); 30 (VAE)
Loss	Binary cross-entropy (AE variants); BCE + KL (VAE)
Random seed	42 (fixed for NumPy and Keras)
Denoiser training noise	Gaussian $\sigma = 0.2$; salt-and-pepper $p = 0.10$
Denoiser test noise levels	$\sigma \in \{0.1, 0.2, 0.3\}$; $p \in \{0.05, 0.10, 0.20\}$
Latent dimensions studied (FC-AE)	$\{2, 8, 16, 32\}$

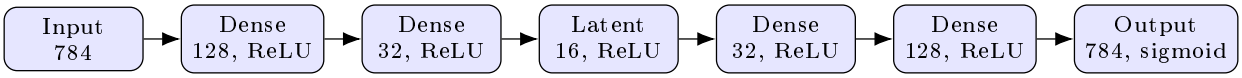
Table 1: Experimental configuration.

Metrics. MSE and MAE are averaged over all pixels of all test images. SSIM is computed per image with `skimage.metrics.structural_similarity` (`data_range`$= 1$) and averaged. The per-image error is $e_i = \frac{1}{784}\sum_j (x_{ij} - \hat{x}_{ij})^2$.

4. Model Architectures

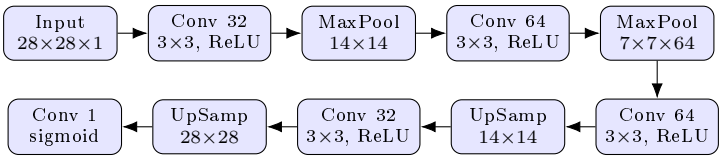


4.1 Fully Connected Autoencoder (FC-AE)



The model has 211,040 parameters. The same architecture was retrained with latent sizes $2, 8, 16, 32$ for the latent-dimension study; $d_z = 16$ is the reference FC-AE.

4.2 Convolutional Autoencoder (CAE)

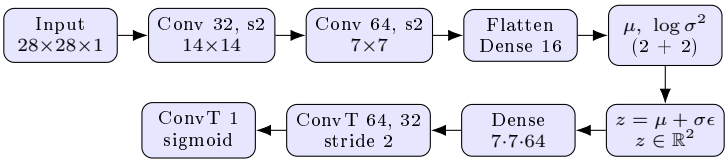


All convolutions use `padding='same'`. The model has 74,497 parameters. The encoder output is a $7 \times 7 \times 64$ feature map (3,136 values). *Note that this is larger than the 784-pixel input, so unlike the FC-AE the CAE does not compress dimensionally; this matters when comparing the two models (Section 6).*

4.3 Denoising CAE (DCAE)

The DCAE uses the same architecture as the CAE (74,497 parameters) but the input is a corrupted image $\tilde{x}$ while the target is the clean image x. Two separate DCAEs were trained, one per noise type (Gaussian $\sigma = 0.2$, clipped to $[0, 1]$; salt-and-pepper $p = 0.10$, half of the corrupted pixels set to 0 and half to 1). The Gaussian DCAE is the one reported in the consolidated tables.

4.4 Variational Autoencoder (VAE)



The encoder produces μ and $\log \sigma^2$, and a custom sampling layer implements the reparameterisation trick $z = \mu + \exp(\frac{1}{2} \log \sigma^2) \odot \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$. The loss is

$$\mathcal{L}_{VAE} = \underbrace{\sum_{p=1}^{784} \mathrm{BCE}(x_p, \hat{x}_p)}_{\mathcal{L}_{rec}} - \tfrac{1}{2} \sum_{j=1}^{2} \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right),$$

with the reconstruction term summed over pixels and both terms averaged over the batch ($\beta = 1$). The model has 134,165 parameters (encoder + decoder). Reconstruction metrics use the posterior mean $z = \mu$.

5. Basic Autoencoder Results (FC-AE, $d_z = 16$)

5.1 Plot 1: Original versus Reconstructed Images

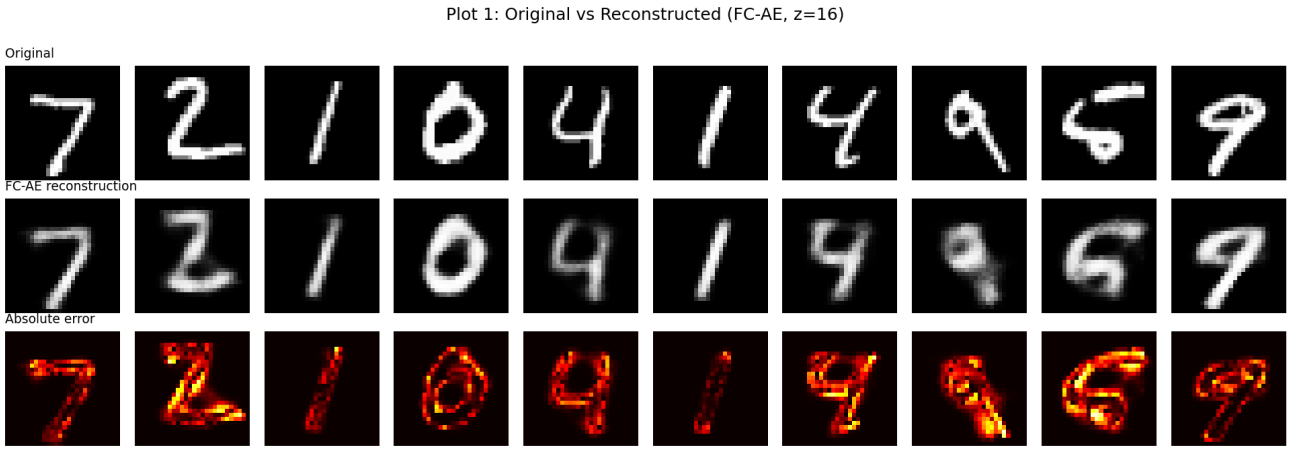


Figure 1: Original, FC-AE reconstruction and absolute error for the first 10 test images.

Inference. *What it shows:* Ten test digits, their FC-AE reconstructions and the absolute-error maps. *Trend:* Simple, thick-stroke digits (7, 1, 0) are reproduced faithfully; complex digits (2, 4, 5, 9 with a thin tail) come back blurred, with softened or partly missing strokes. The error maps concentrate along stroke boundaries and inside loops. *Why:* A 16-value bottleneck cannot store every stroke detail, and fully connected layers treat pixels as an unordered vector, so edges are averaged into smooth blobs. *Conclusion:* Overall digit identity and global shape are preserved, but fine detail is lost and difficult samples are visibly blurred.

5.2 Plot 2: Training and Validation Reconstruction Loss

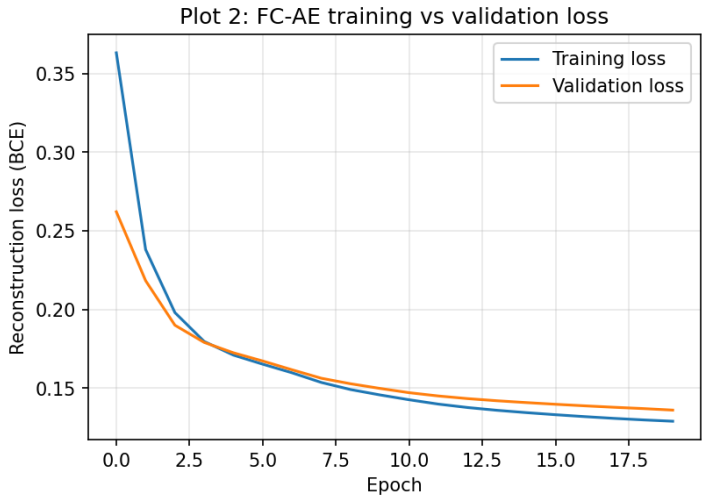


Figure 2: FC-AE training and validation BCE loss over 20 epochs.

Inference. *What it shows:* Training and validation BCE against epoch. *Trend:* Both curves fall steeply in the first 5 epochs and then flatten slowly; final training loss is 0.1290 and final validation loss 0.1361 (gap ≈ 0.007). Validation is lower than training in the first epochs. *Why:* The training loss is averaged over an epoch while the model is still improving quickly, whereas validation is measured at the end of the epoch. The small final gap indicates little memorisation of the 9,000 training images. *Conclusion:* The model converges without clear evidence of overfitting; the curves are still decreasing slowly at epoch 20, so a longer schedule could improve it slightly.

5.3 Reconstruction Metrics

Metric (test set)	Value
Test MSE	0.02388
Test MAE	0.06189
Mean SSIM	0.7161

Table 2: FC-AE ($d_z = 16$) reconstruction metrics on 2,000 test images.

6. Fully Connected versus Convolutional Autoencoder

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.02388	0.06189	0.7161	211,040
Convolutional AE	0.00275	0.01547	0.9728	74,497

Table 3: FC-AE versus CAE on the same test images.

Plot 3: Reconstruction Comparison

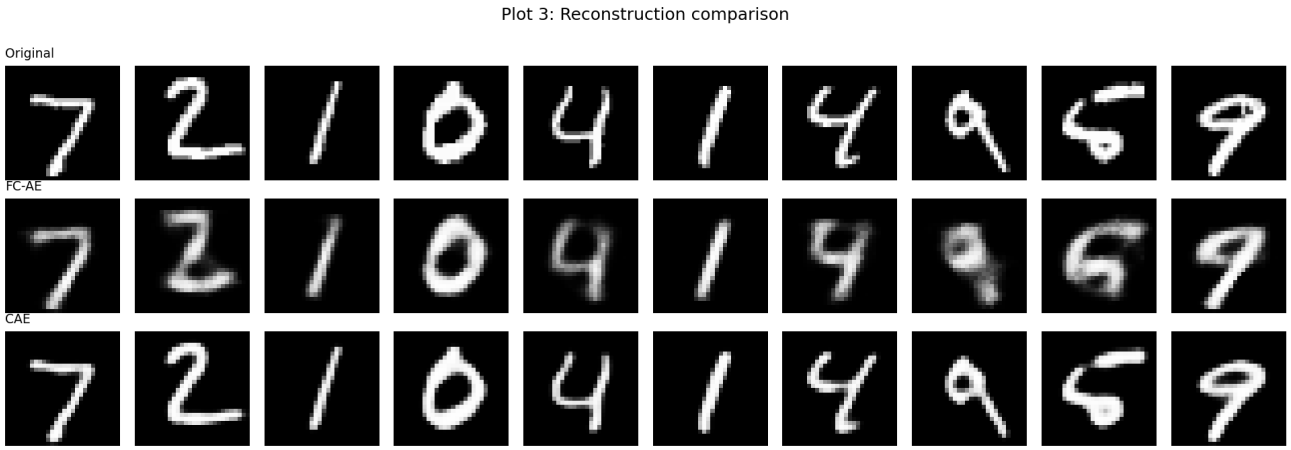


Figure 3: Original, FC-AE and CAE reconstructions of the first 10 test images.

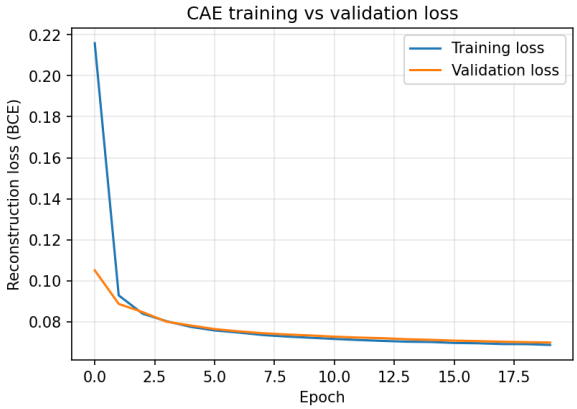


Figure 4: CAE training and validation BCE loss (final: 0.0688 / 0.0699).

Inference. *What it shows:* The same 10 digits reconstructed by the FC-AE and the CAE. *Trend:* The CAE reproduces stroke thickness, closed loops and thin tails almost exactly, while the FC-AE output is smoothed. The CAE reduces MSE by about 8.7×, MAE by 4.0×, and raises SSIM from 0.716 to 0.973, using 65% fewer parameters. *Why:* Convolutions share small local filters across the image, exploit the strong local correlation of neighbouring pixels and keep the spatial layout through pooling and upsampling; a dense layer must learn every pixel relationship separately. *Conclusion:* Preserving spatial locality strongly improves reconstruction. *Caveat:* the CAE latent map (3,136 values) is about 196× larger than the 16-value FC-AE latent, so part of the gain reflects latent capacity and not only convolutional structure; the comparison is therefore architecture-plus-capacity, not a like-for-like compression comparison.

7. Denoising Autoencoder

7.1 Plot 4: Clean versus Noisy versus Denoised

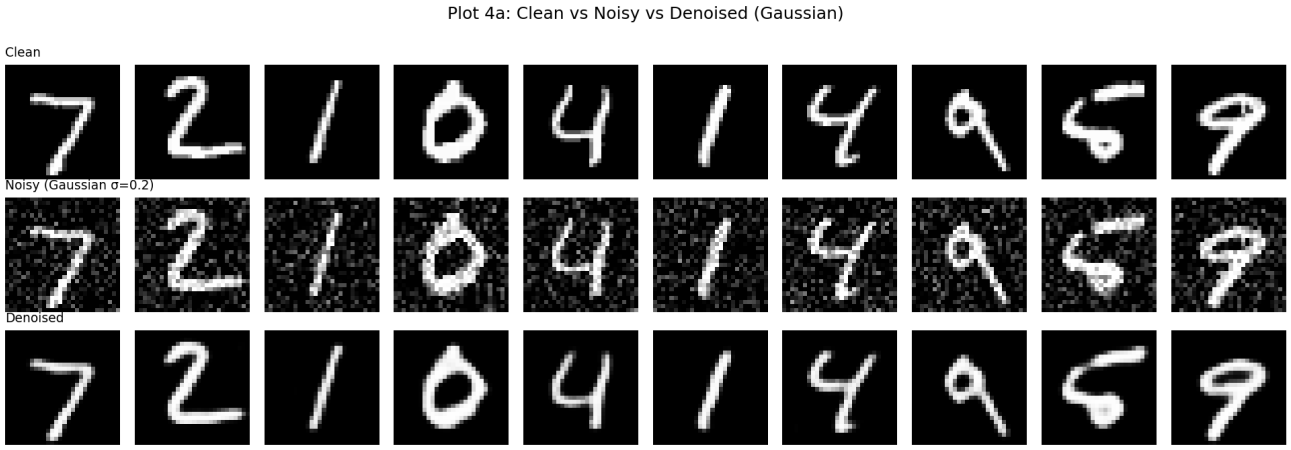


Figure 5: Plot 4a: clean, Gaussian-noisy ($\sigma = 0.2$) and denoised images.

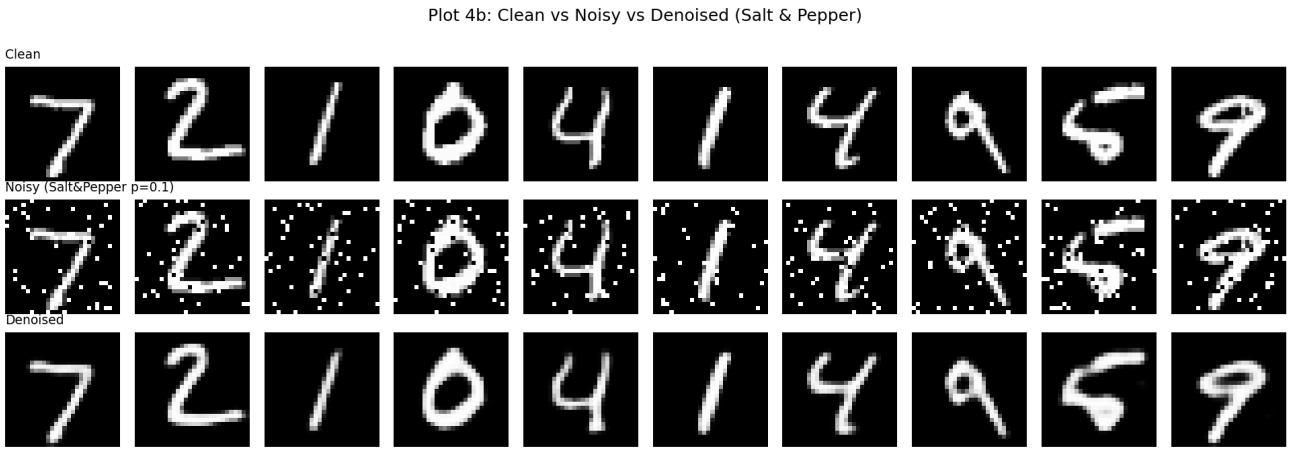


Figure 6: Plot 4b: clean, salt-and-pepper-noisy ($p = 0.10$) and denoised images.

Inference. *What it shows:* Clean digits, their corrupted versions and the DCAE output for two noise types. *Trend:* Under both noise types the isolated noisy pixels disappear and the digit strokes are restored; the outputs are slightly smoother than the clean originals and a few thin details (for example the small hook of the 5) are softened. *Why:* The network is trained to map corrupted inputs to clean targets, so it learns the manifold of digit-like images and treats random pixel noise, which has no consistent structure, as something to discard. *Conclusion:* The DCAE recovers the major structure of the original image for both Gaussian and salt-and-pepper corruption.

7.2 Plot 5: Noise Level versus Reconstruction Quality

All metrics are measured against the clean image, for the noisy input itself and for the denoised output. Each DCAE was trained at a single noise level and evaluated at all three.

		Noisy input			Denoised output		
Noise type	Level	MSE	MAE	SSIM	MSE	MAE	SSIM
Gaussian	$\sigma = 0.1$	0.0054	0.0440	0.6791	0.0036	0.0176	0.9603
Gaussian	$\sigma = 0.2$	0.0212	0.0866	0.5938	0.0046	0.0209	0.9455
Gaussian	$\sigma = 0.3$	0.0466	0.1279	0.5093	0.0069	0.0286	0.8739
Salt & Pepper	$p = 0.05$	0.0241	0.0250	0.7092	0.0043	0.0196	0.9540
Salt & Pepper	$p = 0.10$	0.0479	0.0498	0.5791	0.0052	0.0220	0.9407
Salt & Pepper	$p = 0.20$	0.0968	0.1004	0.4456	0.0085	0.0300	0.8819

Table 4: Noise level versus MSE, MAE and SSIM (test set, 2,000 images).

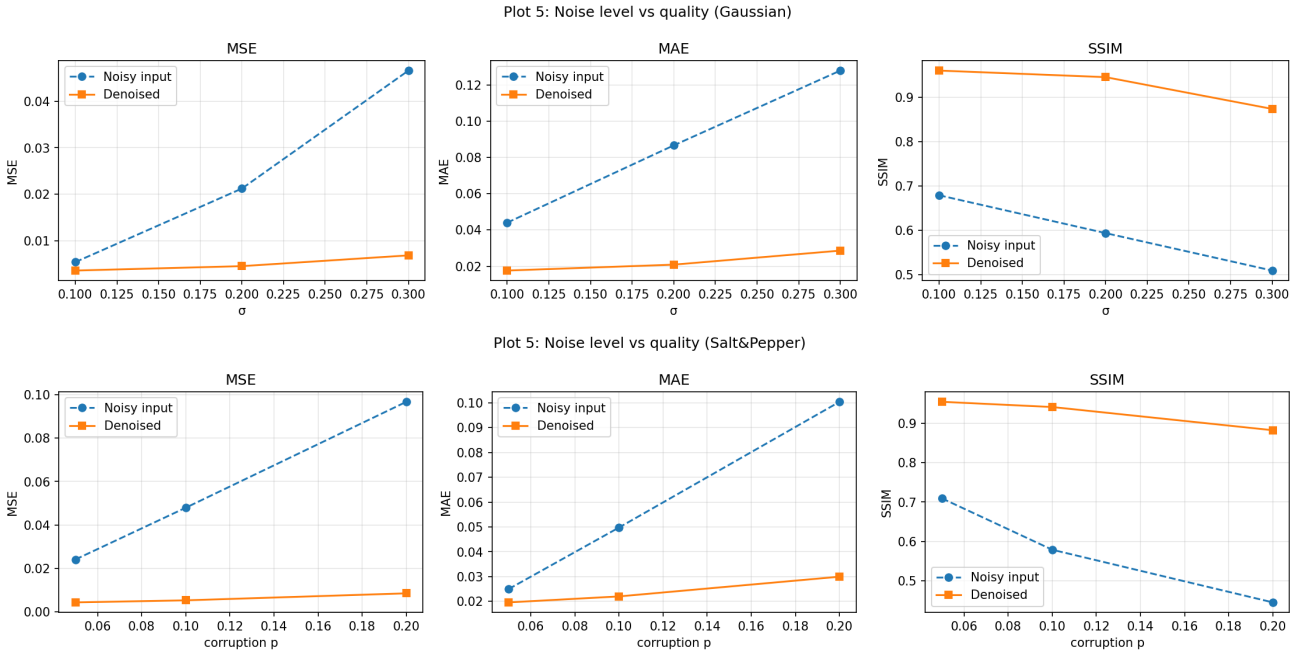


Figure 7: Plot 5: MSE, MAE and SSIM against noise level for Gaussian (top) and salt-and-pepper (bottom) noise. Each metric has its own axis.

Inference. *What it shows:* Reconstruction quality of the noisy input and of the denoised output as corruption increases. *Trend:* As σ rises from 0.1 to 0.3, the noisy-input MSE grows by 8.6× (0.0054 to 0.0466) and its SSIM falls to 0.509, while the denoised MSE grows only 1.9× (0.0036 to 0.0069) and SSIM decreases from 0.960 to 0.874. Salt-and-pepper behaves the same way (denoised SSIM 0.954 to 0.882 for $p = 0.05$ to 0.20). *Why:* Stronger corruption removes more of the information about the underlying stroke, so the restoration problem becomes harder; the network's learned prior on digit shape still lets it recover most of the structure. *Conclusion:* Quality degrades gracefully with noise, and the DCAE recovers the major structure at every level tested, including levels it was not trained on. At the lowest level ($\sigma = 0.1$) the denoised MSE (0.0036) is still above the clean-input CAE (0.0027), consistent with a model tuned for stronger noise smoothing some detail.

7.3 Denoising CAE in the model comparison

For the consolidated table the Gaussian DCAE is evaluated on the $\sigma = 0.2$ noisy test images against the clean targets: MSE 0.00455, MAE 0.02089, SSIM 0.9455, 74,497 parameters.

8. Variational Autoencoder ($z \in \mathbb{R}^2$)

8.1 Loss and Metrics

VAE metric (test set, $z = \mu$)	Value
Reconstruction loss (BCE, summed over pixels)	157.51
KL loss	5.22
Total loss	162.74
Test MSE	0.04734
Test MAE	0.11160
Mean SSIM	0.4259

Table 5: VAE test metrics. Losses are in nats per image.

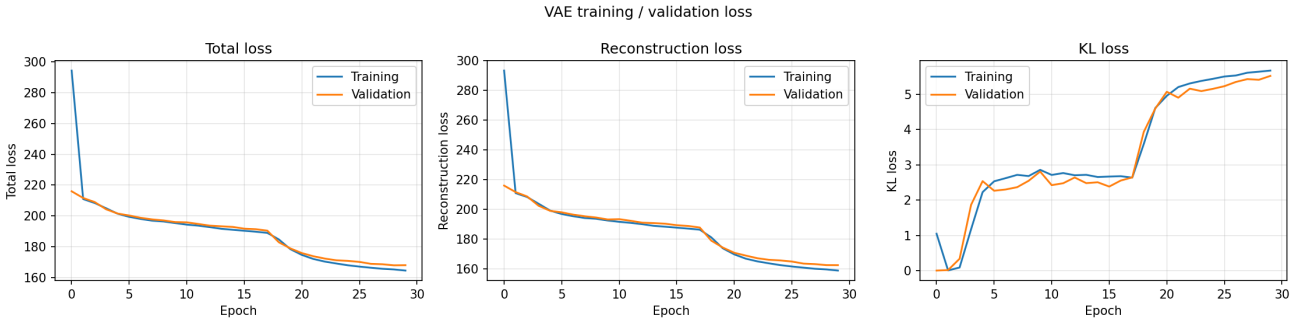


Figure 8: Plot 9 (VAE): training and validation total, reconstruction and KL loss over 30 epochs.

Inference. *What it shows:* Total, reconstruction and KL loss for training and validation data. *Trend:* Total loss falls from about 294 to 164 (training) and 168 (validation). The KL term rises from near zero to about 2.6 and then jumps to about 5.5 after epoch 17, coinciding with a second sharp drop in the reconstruction loss. Training and validation curves stay close (164.4 versus 167.9 at epoch 30). *Why:* Early on the KL penalty keeps the posterior close to the prior, so the latent carries little information; once the encoder begins to use the two latent dimensions more informatively, KL increases and reconstruction improves. This is the reconstruction-versus-regularisation trade-off of the VAE objective. *Conclusion:* Training is stable and shows no overfitting (small train–validation gap), but the losses were still decreasing at epoch 30, so the VAE is probably under-trained. The test reconstruction loss (157.5) is computed with $z = \mu$, whereas the training and validation values use a sampled z, so they are not strictly comparable.

8.2 Plot 6: Latent Space

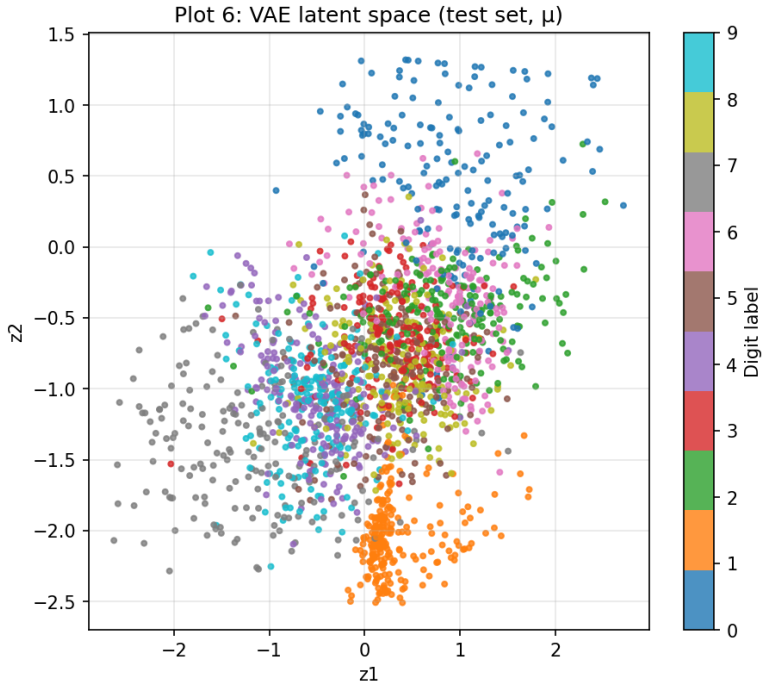


Figure 9: Plot 6: encoded means μ of the test images, coloured by digit label.

Inference. *What it shows:* The 2-D posterior means of all test images, coloured by digit (labels not used in training). *Trend:* The latent cloud is centred near the origin and spans roughly $[-2.7, 2.7]$ on both axes, close to the $\mathcal{N}(0, I)$ prior. Digit 1 forms a tight cluster at the bottom ($z_2 \approx -2.5$ to -1.5), digit 0 occupies the top ($z_2 > 0.4$), and digit 7 lies to the left. Digits 4 and 9 overlap strongly, and 2, 3, 5, 6 and 8 overlap heavily in the centre. *Why:* The VAE is trained without labels and the KL term pulls all encodings towards a single Gaussian blob. With only two dimensions, digits that share shape features (4/9, 3/5/8) cannot be separated. *Conclusion:* Same-digit samples do occupy similar regions, most clearly for 0, 1 and 7, but the classes overlap considerably, so a 2-D latent space captures only coarse digit identity.

8.3 Plot 7: Randomly Generated Images

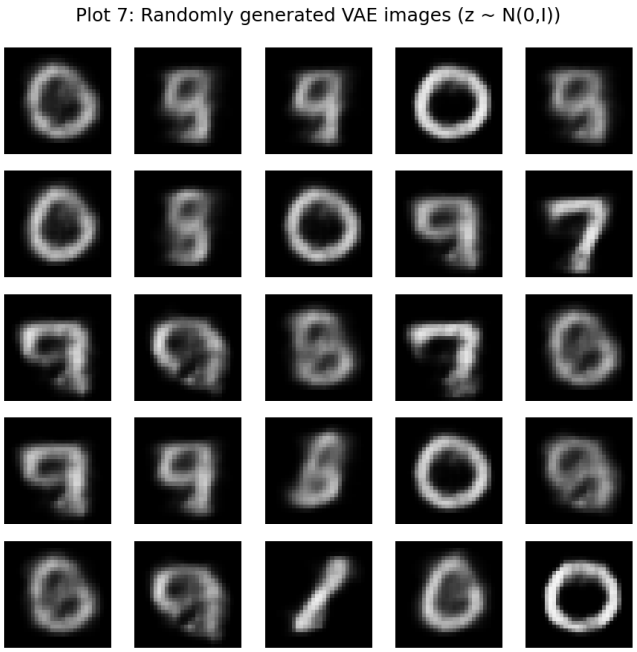


Figure 10: Plot 7: 25 images decoded from $z \sim \mathcal{N}(0, I)$.

Inference. *What it shows:* 25 images generated by decoding random latent samples. *Trend:* Several samples are clearly recognisable (0, 9, 7, 1), but many are blurry, and shapes such as 8-like or 5-like forms are ambiguous. Diversity is limited in this sample: 0-like and 9-like shapes dominate. *Why:* The decoder returns an average of the training images that map near each latent point; the BCE objective favours blurry means, and a 2-D space forces many digits to share regions, producing in-between shapes. *Conclusion:* The VAE can generate digit-like images from noise, but at $d_z = 2$ the samples are blurry, only partly diverse and sometimes ambiguous.

8.4 Plot 8: Latent Space Interpolation

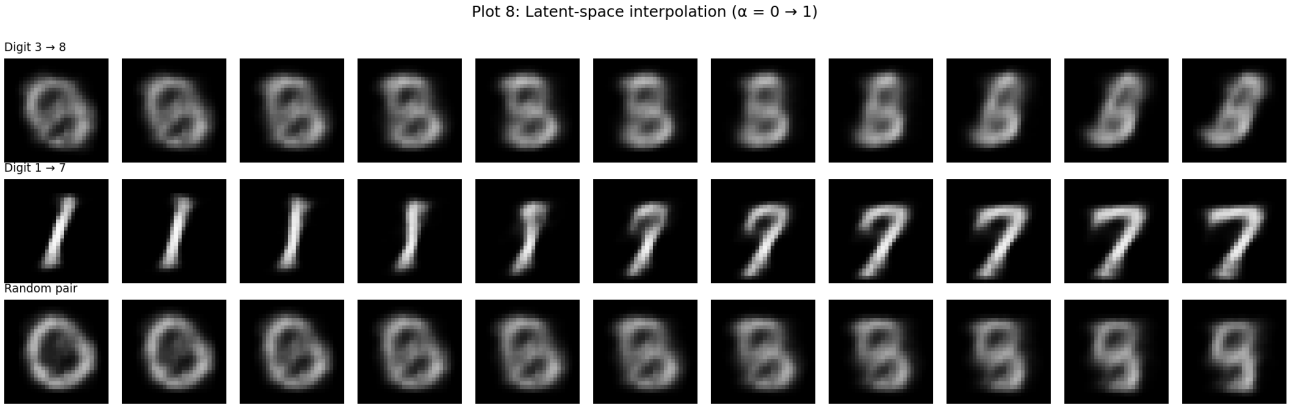


Figure 11: Plot 8: decoded images for $\alpha = 0, 0.1, \ldots, 1$ between two latent points (digit $3 \to 8$, digit $1 \to 7$, and a random pair).

Inference. *What it shows:* Decoded images along the straight line $z(\alpha) = (1-\alpha)z_A + \alpha z_B$. *Trend:* The image changes gradually with α in all three rows; the $1 \to 7$ row is the clearest, with the top hook and bar of the 7 growing steadily. The $\alpha = 0$ endpoint of the $3 \to 8$ row decodes to an 8/0-like shape rather than a clean 3. *Why:* The KL regularisation makes the latent space continuous and densely populated, so nearby latent points decode to similar images. The unclear 3 is consistent with the heavy class overlap seen in Plot 6. *Conclusion:* Interpolation is smooth, showing that the VAE learns a continuous latent manifold rather than isolated codes.

9. Reconstruction Error Distribution

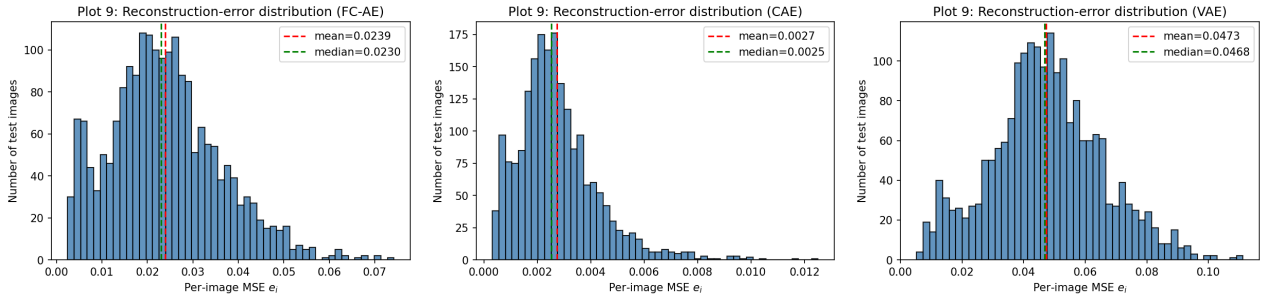


Figure 12: Plot 9: histograms of per-image error e_i on the test set for FC-AE, CAE and VAE. Note the different x-axis ranges.

Model	Mean e_i	Median	Std. dev.	5–95% range	Max
FC-AE	0.0239	0.0230	0.0120	0.0053–0.0454	0.0742
CAE	0.0027	0.0025	0.0015	0.0007–0.0055	0.0125
VAE	0.0473	0.0468	0.0179	0.0151–0.0786	0.1112

Table 6: Per-image reconstruction error statistics on 2,000 test images.

Inference. *What it shows:* How the per-image MSE is spread over the test set for each model. *Trend:* Typical errors lie in 0.005–0.045 (FC-AE), 0.0007–0.0055 (CAE) and 0.015–0.079 (VAE). Mean and median are almost equal, so the bulk is roughly symmetric, with a mild right tail (the largest error is about 3× the mean for the FC-AE and 4.6× for the CAE). The FC-AE and VAE histograms also show a small extra group at low error. *Why:* Most digits are of similar difficulty, while a few unusual or complex shapes are reconstructed much worse. (The low-error group is probably thin-stroke digits such as 1, which have few foreground pixels; this was not checked per class.) *Conclusion:* Errors are concentrated rather than widely spread for the CAE, and the CAE's worst image (0.0125) is still better than the FC-AE's average (0.0239), so lower average error goes together with visibly cleaner reconstructions (Plot 3).

10. High-Error Image Analysis

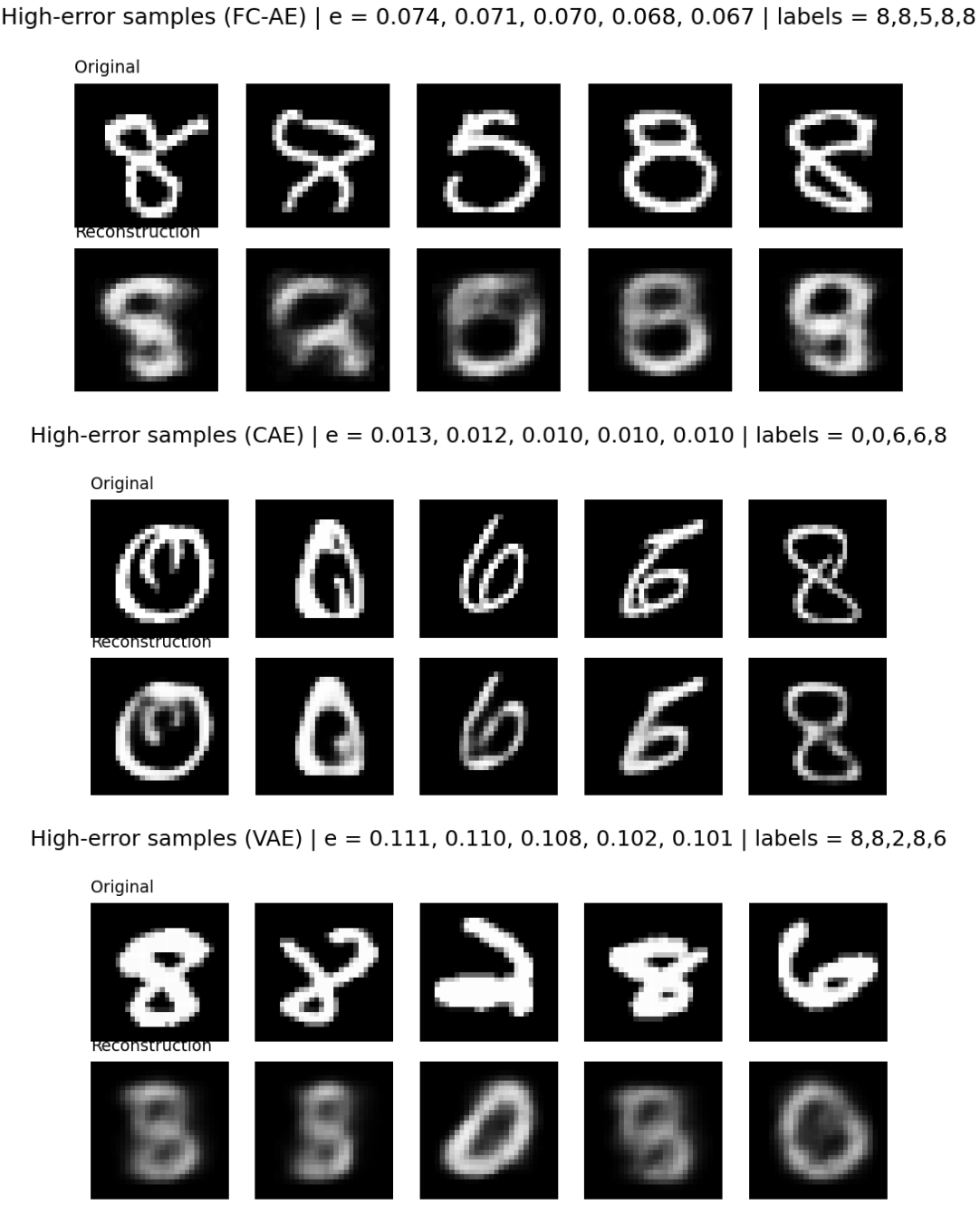


Figure 13: The five test images with the largest reconstruction error for the FC-AE, CAE and VAE (original on top, reconstruction below). The titles give the error values and digit labels.

Model	Errors e_i (top 5)	Digit labels
FC-AE	0.074, 0.071, 0.070, 0.068, 0.067	8, 8, 5, 8, 8
CAE	0.013, 0.012, 0.010, 0.010, 0.010	0, 0, 6, 6, 8
VAE	0.111, 0.110, 0.108, 0.102, 0.101	8, 8, 2, 8, 6

Table 7: Highest-error test samples.

Explanation. For the FC-AE and VAE the hardest images are mostly 8s (and a 5, a 2 and a 6) with crossing strokes, small loops, unusual slants or very thick or thin strokes; the low-capacity models replace them with a blurry average 8 or 0 (the VAE turns the 2 into a 0-like shape). The CAE's worst cases are irregular 0s and 6s with extra internal strokes or open loops, which it reconstructs with the correct outline but smoothed interiors. Hard samples are therefore those that deviate from the typical shape of their class and contain many fine strokes.

11. Latent-Dimension Study (FC-AE)

Latent dim.	MSE	SSIM	Parameters
2	0.04773	0.4260	210,130
8	0.03042	0.6417	210,520
16	0.02388	0.7161	211,040
32	0.02063	0.7525	212,080

Table 8: Effect of latent dimension on reconstruction (test set).

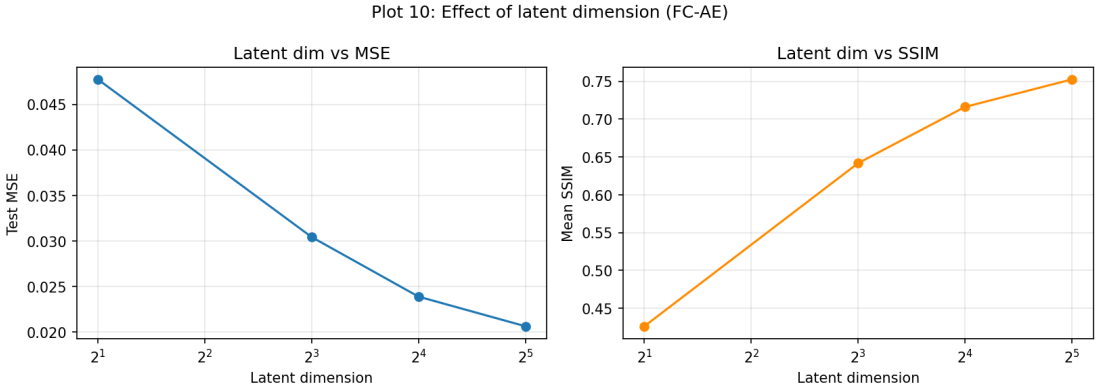


Figure 14: Plot 10: latent dimension versus test MSE (left) and SSIM (right).

Inference. *What it shows:* Test MSE and SSIM of the same FC-AE with latent size 2, 8, 16 and 32. *Trend:* MSE decreases and SSIM increases monotonically with latent size: MSE drops 36% from $d_z = 2$ to 8, 22% from 8 to 16, and 14% from 16 to 32, while SSIM rises from 0.426 to 0.753. *Why:* A larger bottleneck can retain more information about each image, but the marginal benefit shrinks because most of the variation in MNIST digits is captured by a few dozen degrees of freedom. Parameter count barely changes (under 1%) since the encoder and decoder hidden layers dominate. *Conclusion:* There is a compression–quality trade-off: very small latents lose substantial detail, and returns diminish beyond about 16 dimensions, so $d_z = 16$ is a reasonable balance for this setup.

12. Consolidated Results

Model	MSE	MAE	SSIM	Parameters	Time (s)
FC Autoencoder ($d_z = 16$)	0.02388	0.06189	0.7161	211,040	10.6
Conv. Autoencoder	0.00275	0.01547	0.9728	74,497	18.8
Denoising CAE ($\sigma = 0.2$)	0.00455	0.02089	0.9455	74,497	16.5
VAE ($d_z = 2$, $z = \mu$)	0.04734	0.11160	0.4259	134,165	26.8

Table 9: All models on the same 2,000 test images. The DCAE row compares its output on $\sigma = 0.2$ noisy inputs with the clean image. Time is total training time on a Kaggle Tesla T4 GPU; the VAE was trained for 30 epochs and the others for 20, so times are not strictly comparable.

Mandatory inferences (summary).

- **Latent dimension:** a larger latent lowers MSE and raises SSIM with diminishing returns (Table 8).
- **FC versus convolutional:** the CAE has 8.7× lower MSE and much higher SSIM with fewer parameters, although its latent map is far larger than the FC-AE's (Section 6).
- **Gaussian noise level:** denoised MSE rises only 1.9× from $\sigma = 0.1$ to 0.3 while the noisy-input MSE rises 8.6× (Table 4).
- **Denoising ability:** major digit structure is recovered for both noise types (Plot 4); denoised SSIM stays above 0.87 at the highest levels.
- **VAE latent distribution and 2-D structure:** encodings occupy the prior's range, with well-separated 0, 1 and 7 but strong overlap among 2, 3, 4, 5, 6, 8 and 9 (Plot 6).
- **Generated samples and interpolation:** samples are recognisable but blurry and of limited diversity; interpolation is smooth (Plots 7–8).
- **Error versus visual quality:** the models with lower MSE and higher SSIM produce visibly sharper reconstructions, and the highest-error images are complex or irregular digits.
- **Deterministic versus probabilistic latent:** at the same latent size 2, the VAE (MSE 0.0473, SSIM 0.4259) reconstructs almost identically to the deterministic FC-AE (MSE 0.0477, SSIM 0.4260), so the poor VAE reconstruction comes mainly from the 2-D bottleneck and not from the probabilistic formulation. What the VAE adds is a regularised, continuous latent space that supports sampling and interpolation, which a plain autoencoder does not guarantee.

13. Discussion Questions

1. **Autoencoder:** a neural network trained to reproduce its input at the output through a compact intermediate representation.
2. **Encoder:** maps the input to a lower-dimensional latent representation.
3. **Decoder:** reconstructs the input from the latent representation.
4. **Latent representation:** the compressed code (here, e.g., 16 values) that retains the information needed for reconstruction.
5. **Why smaller than the input:** the bottleneck forces the network to learn the structure of the data instead of copying it. (The CAE here is an exception: its 7×7×64 map is larger than the input, so it relies on convolutional structure rather than a dimensional bottleneck.)
6. **Latent too small:** information is lost and reconstructions are blurry (SSIM 0.426 at $d_z = 2$).
7. **Latent increased:** reconstruction improves (MSE 0.0477 to 0.0206 from 2 to 32) with diminishing returns and less compression.
8. **CAE and spatial information:** shared local filters and pooling/upsampling keep the 2-D layout, whereas flattening discards neighbourhood structure.
9. **AE versus denoising AE:** a DAE receives a corrupted input but is trained to output the clean image.
10. **Clean target:** it makes the network learn the underlying image structure; using the noisy image as the target would teach it to reproduce the noise.
11. **Higher noise:** reconstruction quality falls (Gaussian denoised SSIM 0.960 to 0.874 from $\sigma = 0.1$ to 0.3).
12. **MSE, MAE, SSIM:** MSE penalises large pixel errors, MAE measures average absolute deviation, and SSIM measures structural (perceptual) similarity, which pixel errors can miss.
13. **VAE:** a generative autoencoder whose encoder outputs a distribution over the latent variable, regularised towards a prior.
14. **VAE versus AE:** the VAE encodes to a distribution $\mathcal{N}(\mu, \sigma^2)$, samples from it and adds a KL term; the AE uses a single deterministic vector.
15. **Why a distribution:** it yields a continuous, smooth latent space in which nearby points decode to similar images, enabling generation.

16. **Reparameterisation trick:** $z = \mu + \sigma \odot \epsilon$ moves the randomness into ϵ so gradients can flow through μ and σ.
17. **Why $\epsilon \sim \mathcal{N}(0, I)$:** it is a fixed, parameter-free noise source that, scaled and shifted, gives samples from $\mathcal{N}(\mu, \sigma^2)$.
18. **KL divergence:** it pulls each posterior towards the prior, keeping the latent space compact and samplable, at some cost to reconstruction.
19. **Prior:** the standard normal $\mathcal{N}(0, I)$.
20. **2-D latent space:** it can be plotted directly, as in Plot 6.
21. **Interpolation:** shows that the latent space is continuous and that the decoder changes smoothly between images.
22. **Why a VAE can generate:** the KL term makes the latent codes resemble the prior, so decoding samples from $\mathcal{N}(0, I)$ yields plausible images.
23. **Unclear generated images:** the 2-D bottleneck, the blurring effect of the BCE objective, overlapping digit regions, and samples falling between clusters.
24. **Reconstruction versus generation:** reconstruction decodes the latent code of a given image; generation decodes a new latent sample drawn from the prior.
25. **Unseen test images:** they give an unbiased estimate of generalisation; validation was taken from the training subset so the test set stayed untouched.

14. Conclusion

A fully connected autoencoder with a 16-dimensional latent code reconstructs MNIST digits with moderate quality (SSIM 0.716) and converges without overfitting. A convolutional autoencoder achieves far better reconstruction (SSIM 0.973, 8.7× lower MSE) with fewer parameters, although its latent map is much larger than the FC-AE's code. Trained with clean targets, the denoising CAE removes both Gaussian and salt-and-pepper noise and keeps SSIM above 0.87 even at the strongest corruption tested. The 2-D VAE reconstructs less accurately (SSIM 0.426), but this matches a deterministic autoencoder with the same 2-D latent, and it provides a continuous latent space that supports random generation and smooth interpolation, with the trade-offs of blurry samples and heavily overlapping digit classes. The latent-dimension study shows the compression–quality trade-off, with diminishing returns beyond 16 dimensions. All results come from a single run with seed 42 on 10,000 training and 2,000 test images, so small differences should not be over-interpreted.

15. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes," ICLR, 2014.
3. Pascal Vincent et al., "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion," Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, "MNIST Handwritten Digit Database."
5. TensorFlow Documentation: https://www.tensorflow.org
6. Keras Documentation: https://keras.io
7. scikit-image Documentation: https://scikit-image.org
8. Source code for this experiment: https://github.com/tonypanda7/DeepLearning_Lab